

Homework 1

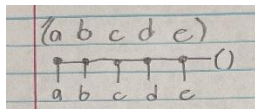
CSC 4101, Fall 2026

Due: 17 September 2026

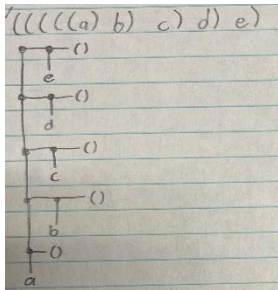
1. (10 pts)

Draw binary tree diagrams for each of the following Scheme expressions:

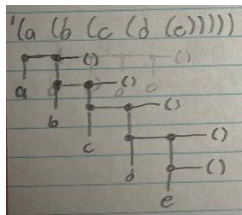
- '(a b c d e)



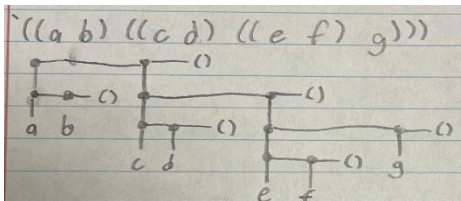
- '((((a) b) c) d) e)



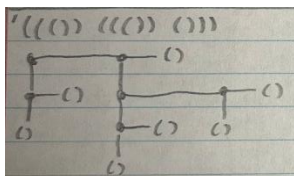
- '(a (b (c (d (e))))))



- '((a b) ((c d) ((e f) g)))



- '(((()) ((()) ()))



2. (10 pts)

Show the result of evaluating each of the following Scheme expressions:

- `(cdr '((a (b c)) (d) (e (f))))`
`((d) (e (f)))`
- `(cons '(a) '(((b) (c))))`
`((a) ((b) (c)))`
- `(car (car (cdr '(a ((b (c) (e (f))))))))`
`b`
- `(quote (car '(a ((b) (c d)))))`
`(car '(a ((b) (c d))))`
- `(car (quote (cons 'a '(((b) (c d))))))`
`cons`

3. (10 pts)

Given the following Scheme definition:

```
(define x '(define (fib n)
  (if (<= n 1) n
      (+ (fib (- n 1)) (fib (- n 2))))))
```

(This does not define the Fibonacci function `fib`, but the variable `x`.)

Write Scheme expressions in terms of `x` that would have the effect of extracting the following expressions:

- `(fib n)`
`(cadr x)`
- `<=`
`(car (cadr (caddr x)))`
- `(fib (- n 1))`
`(cadr (caddr (caddr x)))`
- The third occurrence of `fib`.
`(car (caddr (caddr (caddr x))))`
- The last occurrence of `n`.
`(cadr (cadr (caddr (caddr (caddr x)))))`

E.g., the expression `(car x)` would extract `define`.

4. (10 pts)

Consider the following Scheme function `foo`:

```
(define (foo x)
  (cond
```

```

((null? x) 0)
((not (list? (car x)))
 (+ 1 (foo (cdr x))))
((eq? x '()) (foo (car x)))
((eq? x '(x)) (foo x))
(#t (+ (foo (car x)) (foo (cdr x)))))

```

a. (8 pts)

Explain what the function computes and how. Don't just restate the function definition in English — explain the algorithm. Hint: not all the code in this function does something useful.

It counts the number of atomic elements within the list, regardless of how deeply nested they are. It first tests if the list is empty, then recursively increments the count with each element. If the first element of the list is a sublist, then it calls foo on the first element. The first eq? statement never executes because the list checked for a nil element at the beginning, while the second eq? statement never executes because eq tests pointers to memory, which would never evaluate to true with two separate nodes.

b. (2 pts)

Show the result of executing the expression

```
(foo '(((a b (c d)) (((d e)) f) g)))
```

8

5. (10 pts)

Write a recursive Scheme function with two parameters, an atom and a list, that returns the list with all occurrences, no matter how deep, of the given atom deleted. The returned list cannot contain anything in place of the deleted atoms.

An atom is anything that's not a list, e.g., numbers or symbols. For comparing atoms for equality use eqv?. The pointer equality test eq? is not guaranteed to work for numbers, = only works for numbers. A precise description of these functions can be found in the section 'Equivalence predicates' in the *Revised(5) Report on the Algorithmic Language Scheme*, which you can find by searching for "R5RS" on the web.

```
(define (atom-remove a lst)
```

```
(cond
```

```
((null? lst) '())
```

```
((list? (car lst))
```

```
(cons (atom-remove a (car lst)) (atom-remove a (cdr lst))))
```

```
((eqv? (car lst) a)
```

```
(atom-remove a (cdr lst)))
```

```
(else
```

```
(cons (car lst) (atom-remove a (cdr lst)))))
```